矩阵运算

一、何为矩阵?

矩阵是一种数学概念，它是一个由数字组成的矩形阵列。在阵列中，数字被组织成若干行和列。每个数字都位于特定的行和列上。例如，以下是一个2x2的矩阵（即2行2列的矩阵）：

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

矩阵在数学和科学中有许多重要的应用，它可以用来表示和解决线性方程组。每个方程可以表示为矩阵的一行，其中矩阵的每个元素代表该方程中的系数，如下面的方程组：

$$\begin{aligned} x + 2y &= x' \\ 3x + 4y &= y' \end{aligned}$$

> 一条线我们可以用一元一次方程表示，而在二维平面中的一个形状，我们可以用上面的二元一次方程组表示。之所以出现矩阵，经查阅我比较认同一个观点：数学家想把多元一次方程组抽象成一个一元一次方程，也就是：
>
> f(x) = αx，而前面的系数α就是这里提到的矩阵。

写成矩阵形式则是：

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x' \\ y' \end{bmatrix}$$

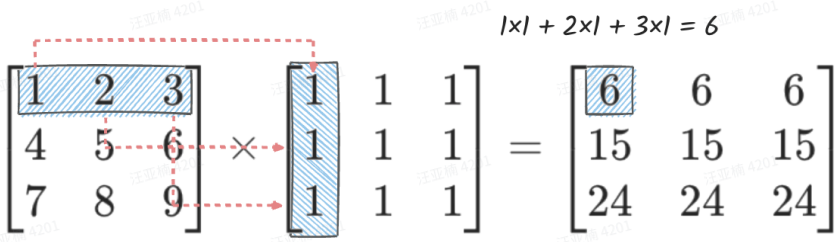
二、矩阵运算

1）矩阵加减

矩阵的加减是非常简单的，两个行列相同的矩阵对应位置的值一一相加减即可，但需要注意的是必须满足行数列数相同。

2）矩阵乘法

矩阵相乘即行列每项一一相乘并相加即为结果矩阵行列相交位置的值，如下图所示：



矩阵乘法是画布变换中的核心，画布变换的效果即是通过矩阵相乘实现，所以请将此牢记于心。

这里需要提醒的几点的是：

① 矩阵的相乘必须满足前一个矩阵的列数等于第二个矩阵的行数。

② 矩阵乘法一般不满足我们曾经所学的乘法交换律，A×B是不一定等于B×A的，如上例矩阵交换相乘：

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} = \begin{bmatrix} ? & ? & ? \\ ? & ? & ? \\ ? & ? & ? \end{bmatrix}$$

解答：

第一行第一列的值为：1×1+1×4+1×7 = 12，其他依此类推，求到最后结果：

$$\begin{bmatrix} 12 & 15 & 18 \\ 12 & 15 & 18 \\ 12 & 15 & 18 \end{bmatrix}$$

可以发现矩阵的顺序不同，结果是不一定相同的。

③ 单位矩阵E是一种特殊的矩阵，**任何矩阵与之相乘结果都是矩阵本身**，3×3的单位矩阵表示如下：

$$E = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

三、`transform(a, b, c, d, e, f)` 的参数含义

该方法是一种将矩阵应用于画布以实现各种变换的强大工具。

在画布绘制中，一个像素原本应该绘制在 `(x, y)` 的位置上，但是调用矩阵变换后，这个点就会映射到另一个位置上 `(x', y')`。

在计算机图形学中，一个像素点（x, y）通常可以用一个2x1的矩阵来表示：

$$\begin{bmatrix} x \\ y \end{bmatrix}$$

当我们调用矩阵变换后，该像素点 `(x, y)` 所表示的矩阵会与画布变换矩阵相乘得到另一个矩阵，而结果矩阵便是映射后的像素位置 `(x', y')` 的矩阵表示：

$$\begin{bmatrix} a & c & e \\ b & d & f \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x' = ax + cy + e \\ y' = bx + dy + f \\ 1 \end{bmatrix}$$

这里之所以将像素点 `(x, y)` 2×1的矩阵改用3×1矩阵，是因为矩阵相乘要求**前一个矩阵的列数等于第二个矩阵的行数**，所以将该2行1列的矩阵改为3行1列齐次坐标矩阵表示。

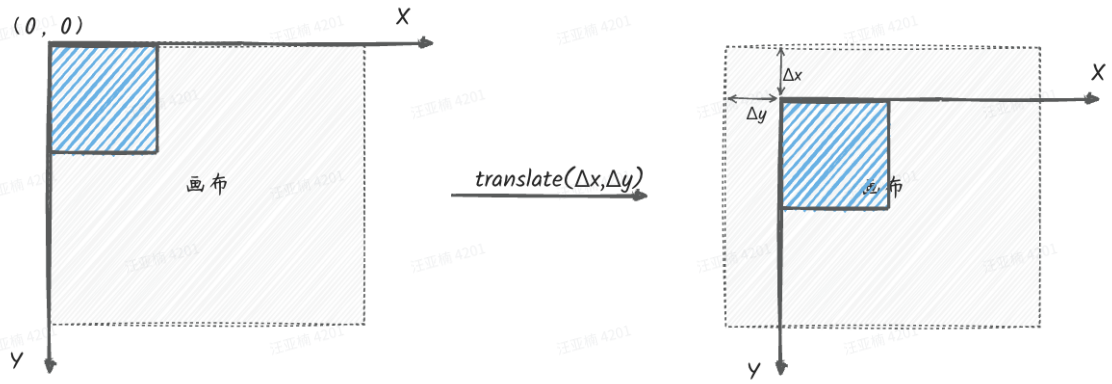
> 齐次坐标是一种在几何和计算机图形学中常用的坐标表示方法，它在常规的笛卡尔坐标（如二维的x和y）基础上，添加了一个额外的维度。
>
> 例如，我们通常使用(x, y)来表示一个点。但在齐次坐标中，我们会使用(x, y, w)来**表示同一个点**，其中w是新增的维度，通常设为1。
>
> 这里不再展开，了解更多请自行查阅。

当我们没有做任何的变换操作时，画布默认的变换矩阵就是上文提到的**单位矩阵E**，对应到上面的参数即是：

```
transform(a = 1, b = 0, c = 0, d = 1, e = 0, f = 0)
```

四、三个原生变换API的底层实现

1） `translate(x, y)` 的底层实现



假设我们调用了 `translate(Δx, Δy)` 将画布原点偏移 `(Δx, Δy)` 的位置，那对应到上面的映射公式中就是：

$$\begin{bmatrix} a & c & e \\ b & d & f \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x+\Delta x \\ y+\Delta y \\ 1 \end{bmatrix}$$

上面我们已经学习了矩阵的相乘了，所以我们可以计算一下左侧的结果：

$$\begin{bmatrix} ax+cy+e \\ bx+dy+f \\ 1 \end{bmatrix} = \begin{bmatrix} x+\Delta x \\ y+\Delta y \\ 1 \end{bmatrix}$$

从上面的公式我们可以得知各个代数符号的值：

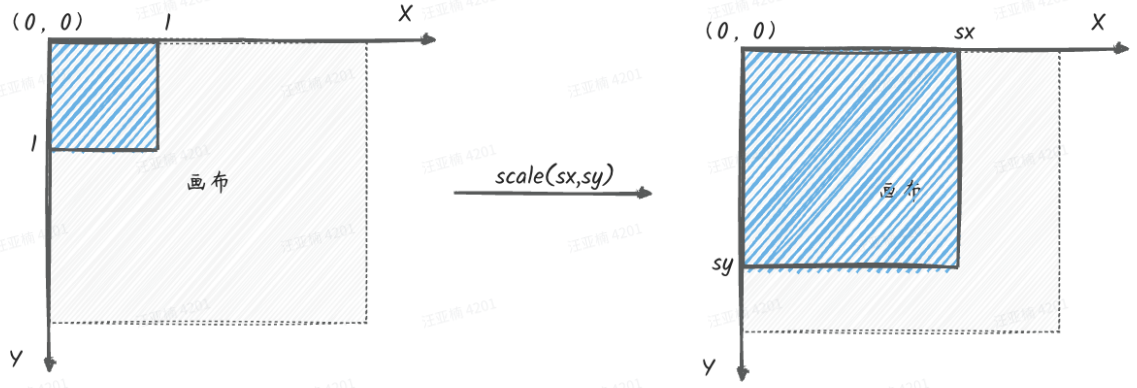
$$a=1、b=0、c=0、d=1、e=\Delta x、f=\Delta y$$

所以 `translate(x, y)` 的底层实现也就是给当前画布偏移的变换矩阵：

$$\begin{bmatrix} 1 & 0 & \Delta x \\ 0 & 1 & \Delta y \\ 0 & 0 & 1 \end{bmatrix}$$

即：`translate(Δx, Δy) = transform(1, 0, 0, 1, Δx, Δy);`

2） `scale(sx, sy)` 的底层实现



当我们调用 `scale(sx, sy)` 时，原本应该绘制在 `(x, y)` 的位置上的点会变成绘制在 `(sx * x, sy * y)` 的位置上。对应到上面的映射公式中：

$$\begin{bmatrix} a & c & e \\ b & d & f \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x * sx \\ y * sy \\ 1 \end{bmatrix}$$

同样的计算左侧矩阵相乘，这里和上面是一样的：

$$\begin{bmatrix} ax + cy + e \\ bx + dy + f \\ 1 \end{bmatrix} = \begin{bmatrix} x * sx \\ y * sy \\ 1 \end{bmatrix}$$

同理可得到各个代数符号的值：

$$a = sx、b = 0、c = 0、d = sy、e = 0、f = 0$$

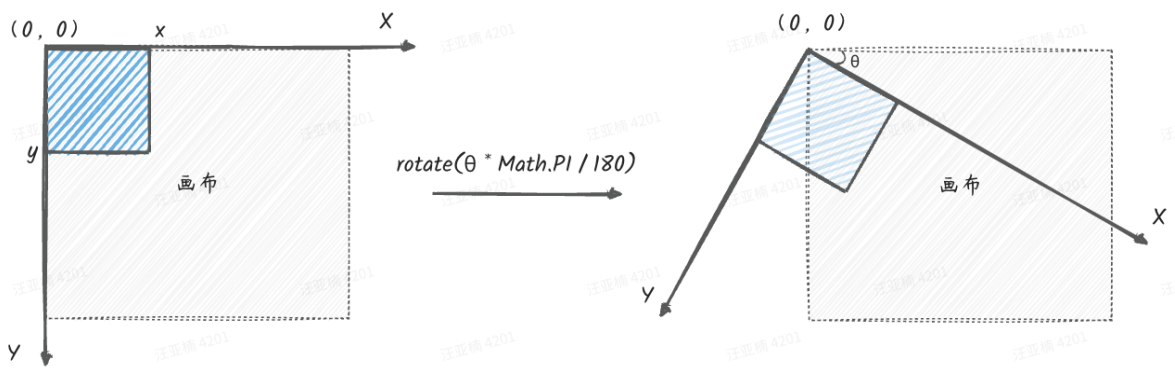
所以 `scale(sx, sy)` 的底层实现也就是给画布追加了缩放的变换矩阵：

$$\begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

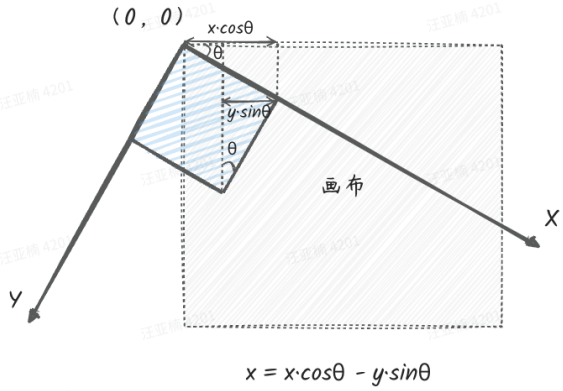
即：`scale(sx, sy) = transform(sx, 0, 0, sy, 0, 0);`

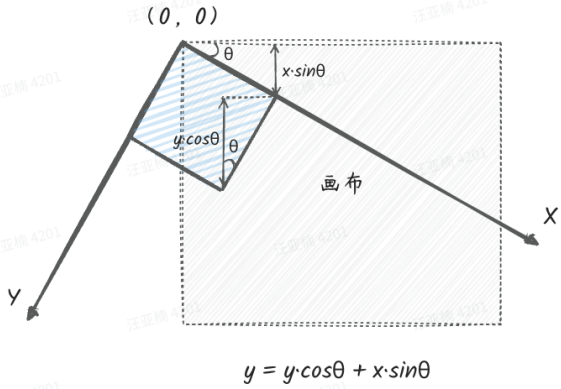
3) `rotate(angle)` 的底层实现

相比偏移和缩放，旋转就略显复杂些了，但是说到底也就是三角函数：



当我们调用 `rotate(angle)` 时，原坐标 `(x, y)` 会被映射到 `(x·cosθ - y·sinθ, y·cosθ + x·sinθ)`，可看下图理解：





和上面的同理，可以得到各个代数符号的值：

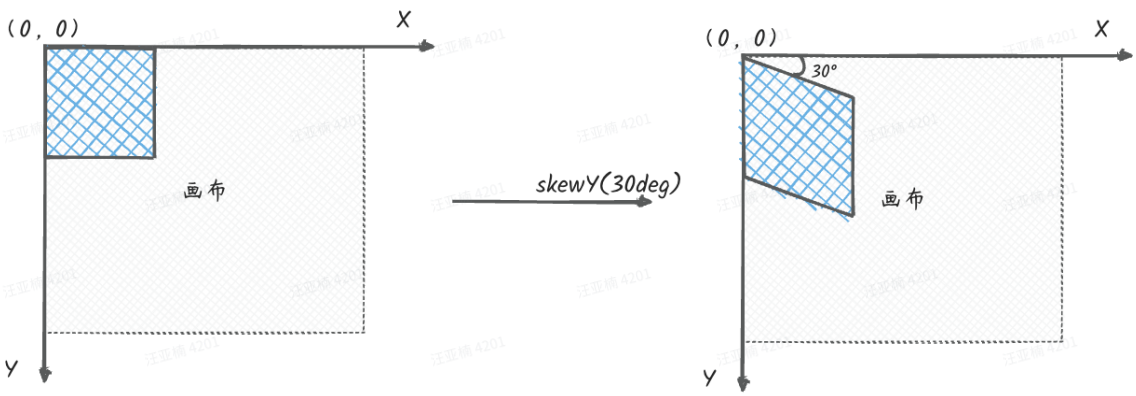
$$a = cos\theta、b = sin\theta、c = -sin\theta、d = cos\theta、e = 0、f = 0$$

所以 `rotate(angle)` 的底层实现也就是给画布追加了缩放的变换矩阵：

$$\begin{bmatrix} cos\theta & -sin\theta & 0 \\ sin\theta & cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

即：`scale(θ * Math.PI / 180) = transform(cosθ, sinθ, -sinθ, cosθ, 0, 0);`

4）倾斜效果（skew）的实现



这里假设只倾斜Y轴30°，原坐标 `(x, y)` 会被映射到 `(x, x * tan30° + y)`：

$$\begin{bmatrix} a & c & e \\ b & d & f \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ x * tan30^{\circ} + y \\ 1 \end{bmatrix}$$

可以得到各个代数符号的值为：

$$a = 1、b = tan30^{\circ}、c = 0、d = 1、e = 0、f = 0$$

X轴倾斜和Y轴倾斜是没什么本质上的区别的，只是变成了：

$$\begin{bmatrix} a & c & e \\ b & d & f \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + y \cdot tan\theta \\ y \\ 1 \end{bmatrix}，c = tan\theta$$

所以当你要设置图形倾斜X轴角度θ，Y轴角度λ时，可以为画布追加变换矩阵设置：

$$\begin{bmatrix} 1 & tan\theta & 0 \\ tan\lambda & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

5）矩阵相关API

① `resetTransform()`，重置矩阵，相当于`transform(1, 0, 0, 1, 0, 0)`。

② `setTransform(a, b, c, d, e, f)`，取消当前变换矩阵，重新设置新的变换矩阵。

6）注意：变换的顺序结果也不同（矩阵相乘不符合交换率）。